

Data Science Colle... · [Follow publication](#)

You're reading for free via [Anubhav's](#) Friend Link. [Upgrade](#) to access the best of Medium.

★ Member-only story

How I Would Become an AI Engineer in 2026 If I Had to Start Over

What I would learn, what I would ignore, what I would build, and how I would know I was actually getting better.

16 min read · Mar 22, 2026



Anubhav

Follow

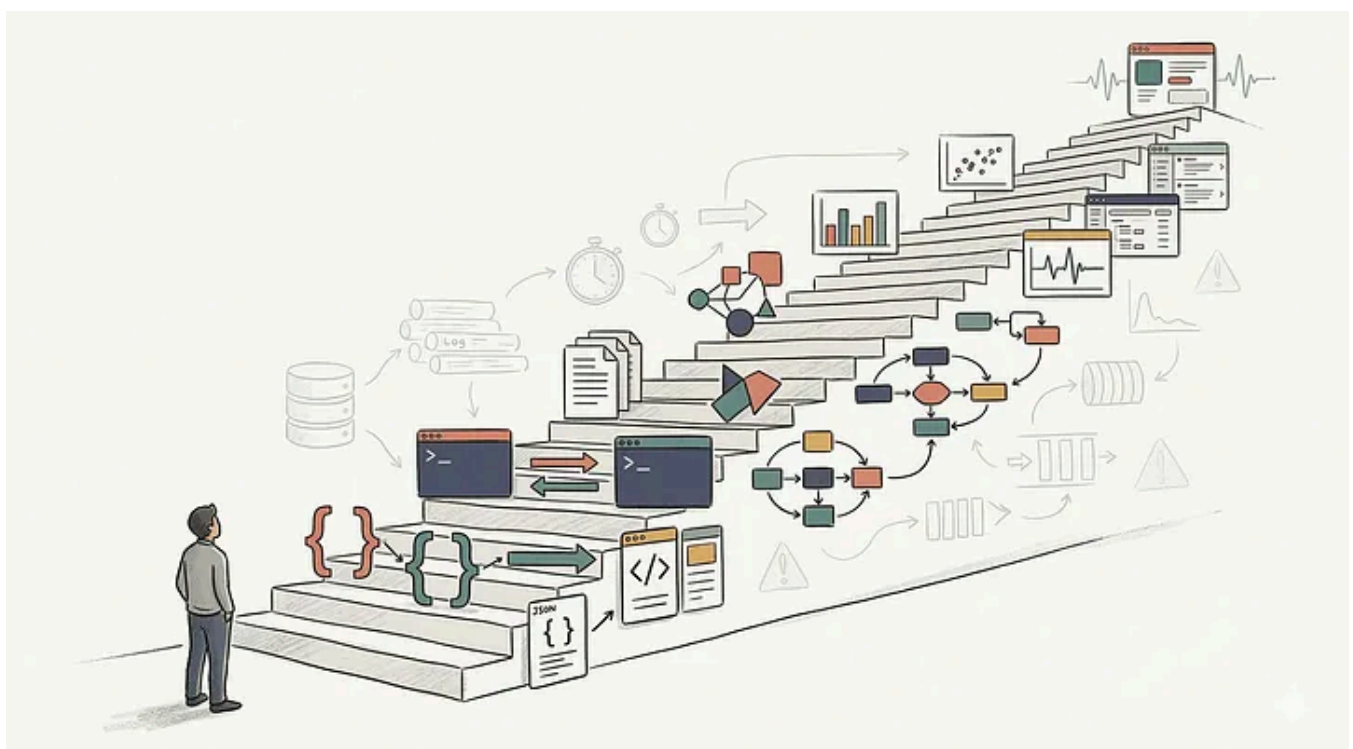


Listen



Share

... More



Most aspiring AI engineers waste months learning the wrong things in the exact wrong order. They start with a forty-hour deep learning course. They spend weeks trying to understand the calculus behind backpropagation. They read endless debates about which vector database is the absolute fastest.

Then they try to build a simple application and they freeze. They do not know how to handle an API timeout. They do not know how to force a model to output valid JSON. They build a chatbot demo in a Jupyter notebook but they have no idea how to put it on a real server.

Not a medium member? Read the full article [here](#).

If you work as an AI engineer in a fast-paced startup today, your reality is very different from the academic roadmaps. You probably do not train transformers from scratch. You build products. You wire up models to databases. You fight with flaky APIs and try to keep AWS bills under control. When you deploy in Mumbai and call a model hosted in Virginia, you deal with network latency before the model even starts generating tokens. That is the reality of the job.

I want to give you a practical roadmap. This is exactly what I would learn, what I would ignore, and what I would build to become useful fast.

What AI Engineers Actually Do In 2026

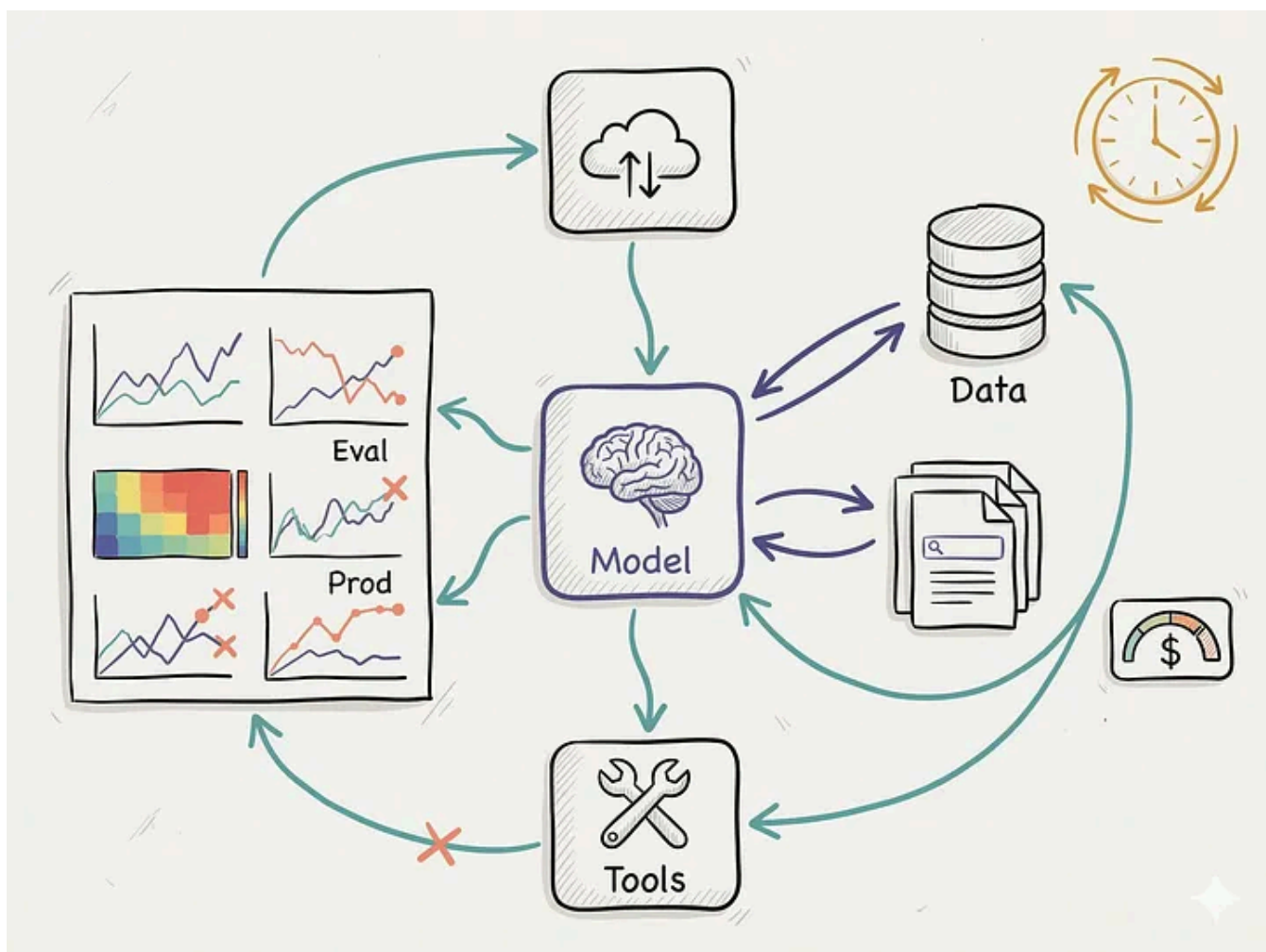
Before you follow any roadmap, you need to understand the actual job. An AI engineer today is usually not a pure machine learning researcher. You are not going to spend your days reading benchmark papers and tweaking custom kernels.

The job is much closer to a mix of product engineering and backend engineering. You are an LLM systems builder.

The actual work looks like this. You call models through APIs. You write very specific instructions to get structured outputs. You build retrieval pipelines to fetch data from your company database. You wire tools together so a model can actually execute actions and spend a lot of time evaluating output quality. You debug failures when the system does something stupid in production and you manage latency and token costs.

If you misunderstand this reality, your learning path will be completely wrong. You do not need to learn all the math behind artificial intelligence to become useful. You

need to learn how to build reliable software that happens to use language models as reasoning engines.



What I Would Ignore At The Beginning

I want to be very clear about what to skip. I am not saying these topics are useless. I am saying they are bad starting points if your goal is to get a job or build a product quickly.

I would deliberately deprioritize training models from scratch. Do not try to build a tiny Llama model in PyTorch on day one. I would also skip advanced fine-tuning. You should build basic applications with standard models first. Fine-tuning is a massive distraction when you do not even know how to prompt correctly.

I would ignore deep GPU internals. You do not need to know how memory is allocated on an Nvidia H100 to build a good support bot. I would ignore the endless vector database comparisons. Just pick pgvector or Pinecone and move on.

I would stop copying random open-source agent repositories without understanding what the code actually does. I would definitely stop doing endless prompt hacking

without a real evaluation framework. Changing a word in your prompt and eyeballing the output is not engineering.

Most importantly, I would not spend months on theory before writing real software.

If your target role is a core ML researcher at OpenAI, this roadmap is not for you. But if you want to be an AI product engineer or a startup backend developer, this is exactly where you should start.

The 4-Stage Roadmap

This is the core progression. Do not skip Stage 1.

Stage 1: Become Dangerous With Code

The very first goal is to stop being blocked by basic software engineering problems. Building AI systems is impossible if building regular software is a struggle.

Start with learning Python. Understand basic APIs and HTTP requests. I see people struggling with model latency all the time because they never bothered to learn basic asynchronous programming. Handling JSON schemas and async calls is mandatory before moving on to complex agent workflows. Writing logs, debugging errors, and actually reading documentation are skills that pay off every single day.

Learning light backend patterns is also part of this stage. This means setting up a simple FastAPI server, using background workers for long tasks, and managing environment variables securely so API keys do not end up on GitHub. Building small API integrations is the best way to practice. Writing a script that calls an API, parses the response, and saves it to a file builds real muscle memory.

This stage is complete when building a small web app end-to-end takes a few hours instead of getting stuck on environment setup for two days.

Here is what dangerous with code actually looks like. It is just a clean FastAPI endpoint that does not crash when the upstream API fails.

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import httpx
import os
import logging

logger = logging.getLogger(__name__)
```

```
app = FastAPI()
http_client: httpx.AsyncClient | None = None

ANTHROPIC_URL = "https://api.anthropic.com/v1/messages"

class QueryRequest(BaseModel):
    user_id: str
    question: str

class QueryResponse(BaseModel):
    content: str

@app.on_event("startup")
async def startup():
    global http_client
    http_client = httpx.AsyncClient(timeout=30.0)

@app.on_event("shutdown")
async def shutdown():
    if http_client:
        await http_client.aclose()

@app.post("/process-query", response_model=QueryResponse)
async def process_query(req: QueryRequest):
    api_key = os.getenv("LLM_API_KEY")
    if not api_key:
        raise HTTPException(status_code=500, detail="Server misconfigured")

    try:
        response = await http_client.post(
            ANTHROPIC_URL,
            headers={
                "x-api-key": api_key,
                "anthropic-version": "2023-06-01",
                "Content-Type": "application/json",
            },
            json={
                "model": "claude-sonnet-4-20250514",
                "max_tokens": 1000,
                "messages": [{"role": "user", "content": req.question}],
            },
        )
        response.raise_for_status()
        data = response.json()
        text = "".join(
            block["text"] for block in data.get("content", []) if block.get("type") == "text"
        )
        return QueryResponse(content=text)

    except httpx.TimeoutException:
        raise HTTPException(status_code=504, detail="Upstream API timed out")
    except httpx.HTTPStatusError as e:
```

```
logger.error("Anthropic API error: %s %s", e.response.status_code, e.re
raise HTTPException(status_code=502, detail="Upstream API error")
except Exception:
    logger.exception("Unexpected error in process_query")
    raise HTTPException(status_code=500, detail="Internal server error")
```

This code is not doing anything magical. It takes a JSON payload and forwards it to Anthropic. But it handles the API key securely. It uses an async client so your server does not block other requests while waiting for the model. It has a strict 30-second timeout. It catches errors and returns proper HTTP status codes. This is the baseline engineering you need before you worry about agents.

Stage 2: Learn LLM App Fundamentals

Now you learn what model-powered apps actually do under the hood.

Prompting basics are the first step, specifically understanding the difference between system instructions and user messages. Forcing structured outputs is the most important skill in this entire stage. Tool calling is another core concept that needs to be mastered, along with a deep understanding of context windows and token limits. Writing retry loops and handling timeouts becomes second nature here.

Building an extraction app is a great way to practice these concepts. A simple assistant that uses a calculator tool or a workflow that turns messy text into clean database rows will expose all the weird quirks of language models.

This stage is finished when making a model produce a reliable JSON object feels routine instead of feeling like a magic trick.

This is what structured output looks like using Pydantic and the OpenAI SDK in 2026. The exact shape of the data is defined upfront.

```
from pydantic import BaseModel, Field
from typing import Literal
from openai import AsyncOpenAI, APITimeoutError, RateLimitError
import asyncio

client = AsyncOpenAI(timeout=30.0)

CATEGORIES = Literal["billing", "technical", "account", "general"]
```

```

class CustomerIssue(BaseModel):
    category: CATEGORIES
    urgency_score: int = Field(ge=1, le=5, description="1=low, 5=critical")
    summary: str = Field(max_length=200)
    needs_human: bool

SYSTEM_PROMPT = """Classify the support ticket. Use these categories: billing,
Rate urgency 1-5 (1=low, 5=critical). Set needs_human=true if the issue involve

async def classify_ticket(text: str) -> CustomerIssue:
    try:
        completion = await client.chat.completions.create(
            model="gpt-4o",
            messages=[
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": text},
            ],
            response_format=CustomerIssue,
        )
        result = completion.choices[0].message.parsed
        if result is None:
            raise ValueError("Model refused or returned unparseable response")
        return result
    except (APITimeoutError, RateLimitError):
        raise
    except Exception as e:
        raise RuntimeError(f"Classification failed: {type(e).__name__}") from e

```

Notice how we define `CustomerIssue` as a Pydantic model. We pass that directly into the API call using `response_format`. This setup gives you a much more reliable path to typed output, but you still need to handle refusals, parse failures, and validation errors.

Stage 3: Learn The Systems Layer

This is where you move from building demos to building systems. ▶

You need to learn Retrieval-Augmented Generation. Understand chunking strategies and retrieval quality. Learn about state machines and workflows. Learn how to persist data, resume processes and how to evaluate your RAG pipeline. You should also learn basic guardrails and observability.

You should build a RAG assistant that actually works. Build a stateful workflow using a framework like LangGraph. Build an eval harness to test your prompts. Build a system that has explicit stop conditions and asks a human for approval.

This stage is complete when explaining how an application fails and exactly how that failure is measured becomes easy.

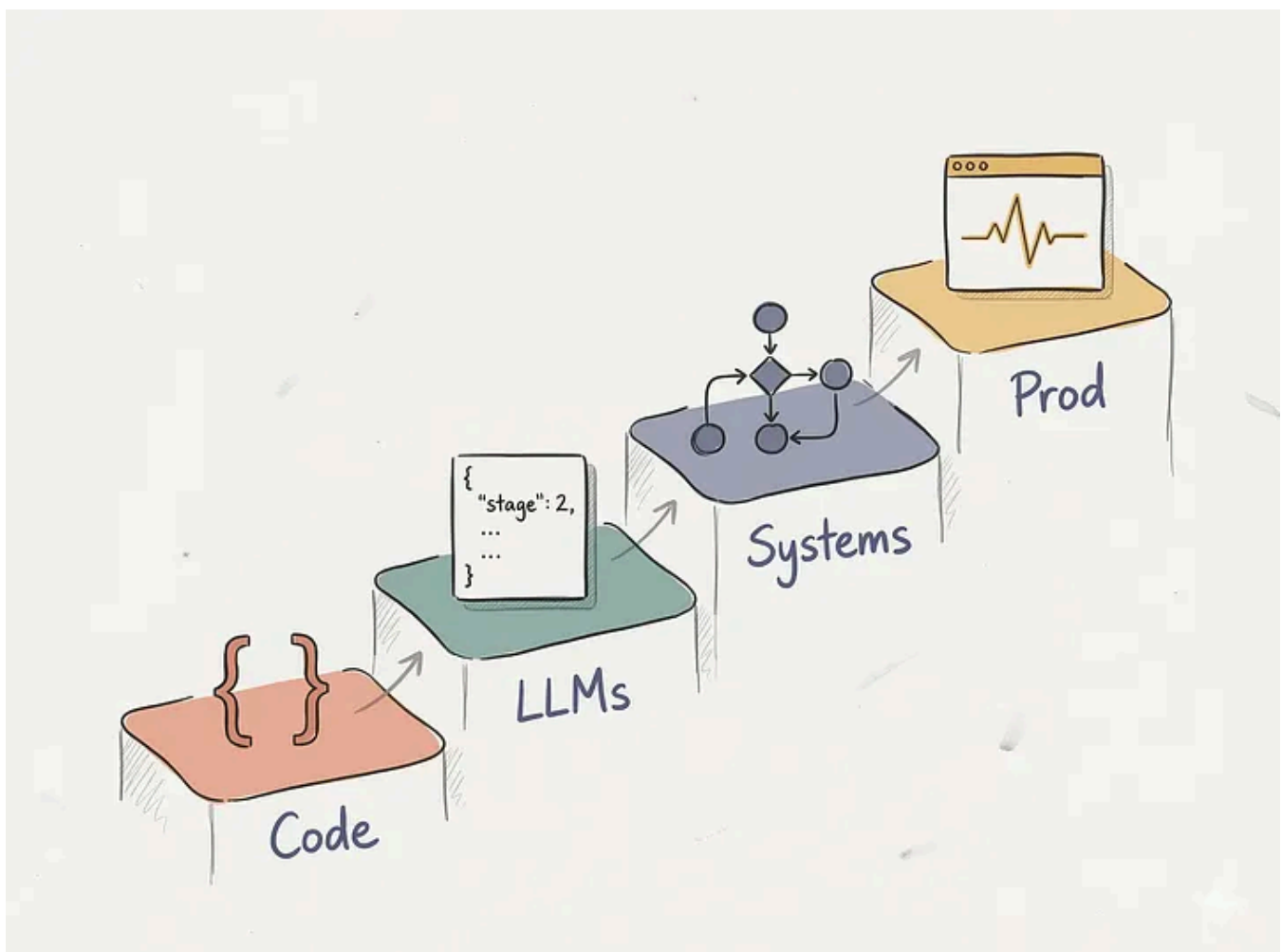
Stage 4: Learn Production Reality

This stage makes you someone a team can actually trust.

You need to learn deployment. Learn how to use queues and background workers. Learn how to monitor your system. Understand cost and latency tradeoffs. You must learn retry and idempotency patterns. You should know how to debug an incident. You need to track product metrics and design a good user experience when the AI inevitably makes a mistake.

Deploying one end-to-end feature to a real server and setting up a monitoring loop is the goal. Write a serious portfolio project with tests and a document explaining the architectural tradeoffs.

This stage is done when talking credibly about production behavior replaces talking about notebook experiments.



The 5 Projects I Would Build

Building a small number of deep projects is much better than building twenty superficial chatbot clones. These five specific projects force the right lessons.

Project 1: Structured Output Extractor

Taking messy real-world data and turning it into clean JSON is the core of AI engineering.

An invoice parser, a tool that reads support tickets and categorizes them, or a contract clause extractor are all perfect examples. This project teaches prompting and how to define strict schemas. It forces the implementation of retry logic for when the model hallucinates a field or returns the wrong data type. Validation is a huge part of this because models sometimes output a string like “100 dollars” when the schema explicitly asked for an integer. Writing code to catch that and ask the model to try again is a great learning experience.

This matters because 80% of enterprise AI work is just turning unstructured text into structured data so traditional software can use it. Mastering this makes someone immediately employable. Spending days writing Pydantic models and testing them against weird edge cases is just part of the job.

Project 2: Retrieval And RAG Assistant

Building an application that searches a dataset and answers questions based strictly on that dataset is a mandatory project.

An assistant for internal company documentation or a search tool for personal notes works perfectly. This teaches chunking in a very practical way. Splitting text by a hard character count usually destroys the meaning of the document, so learning semantic chunking becomes necessary. Embedding models and vector search are introduced here.

Grounding and citation discipline are the most important lessons from this project. Forcing the model to quote its sources is surprisingly difficult. It forces a confrontation with retrieval failures because if the search step returns garbage, the language model will just summarize that garbage beautifully. The model cannot answer a question if the right paragraph is not actually in the context window. Tweaking search queries to make sure the right data gets fetched takes up a lot of time.

Project 3: Tool-Using Workflow

A system where the model actually decides to take an action changes how you think about application design.

A ticket triage workflow that updates a database or a research workflow that searches the web and scrapes websites are great examples. Tool calling and action boundaries are the core concepts here. Handling state transitions and dealing with tool failures are the main challenges. You will see what happens when a tool throws a server error and how the model reacts to that error. Sometimes it tries again and sometimes it just hallucinates a success message.

This project clearly shows the difference between a passive chatbot and an active system.

Project 4: Stateful Agent With Evaluations

An agentic system that runs in a loop but has strict controls is a major step up in complexity.

Open in app ↗

≡ Medium



execution of an agent and wait for human approval. Evaluation thinking is introduced heavily here because testing an agent against a dataset of past incidents is the only way to know if it actually works.

This is the point where building toy agents that get stuck in infinite loops stops and building bounded reliable automation begins.

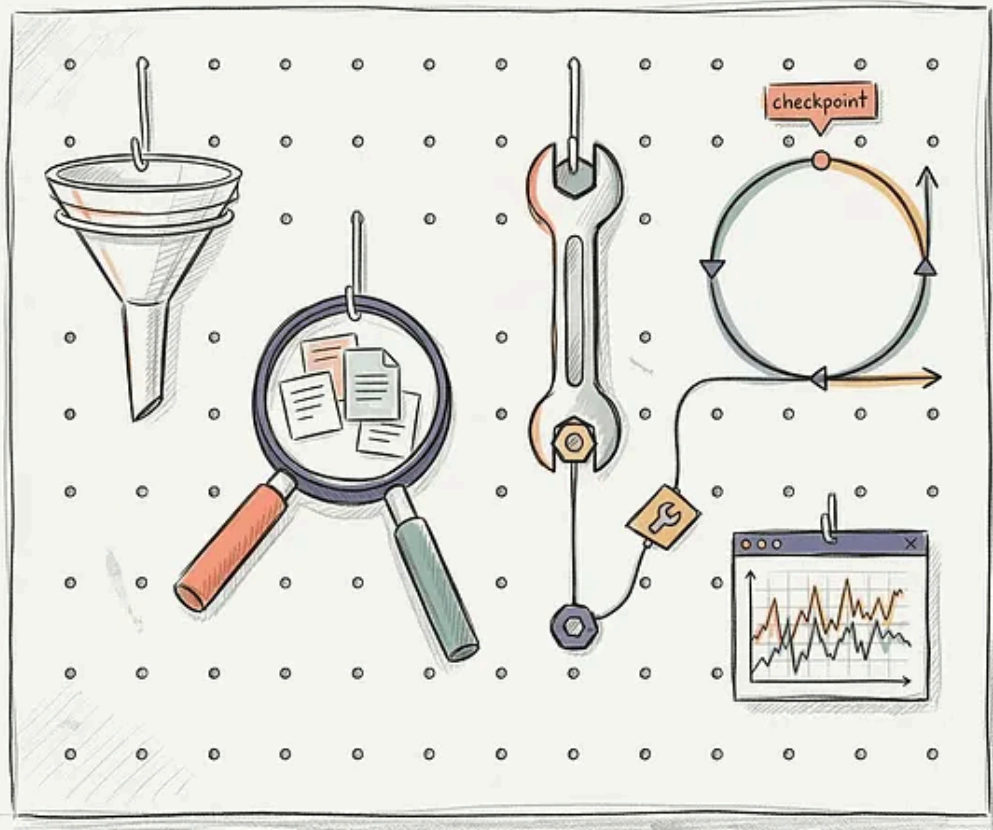
Project 5: One Deployed Mini-Product

Here you can deploy an internal AI assistant or build a small AI feature into a NextJS web app.

This project teaches you deployment. It teaches you telemetry. You will hook up LangSmith or Helicone to trace your API calls. It teaches you cost and latency tradeoffs. You will realize that a chain of four model calls takes 15 seconds and users hate waiting that long. You will learn to stream tokens to the frontend so the user sees text appearing immediately.

It matters because shipping one real thing teaches you more than reading ten architecture threads on social media. You will face CORS errors. You will face rate

limits. You will learn how to secure your API routes so random people do not drain your OpenAI credits.



The 90-Day Plan

I want to give you a timeline. This assumes you are spending a few hours a day after work or classes.

Days 1 to 30

Your focus is Python, APIs, and structured outputs. You need to build two tiny but useful scripts.

Your milestones are simple. Build one extraction tool. Build one structured-output app with schema validation. Publish one clean GitHub repository. Do not worry

about vector databases or agents yet. Just get comfortable calling the APIs and handling the responses. Learn how to read the JSON errors when a model refuses to answer a prompt.

Days 31 to 60

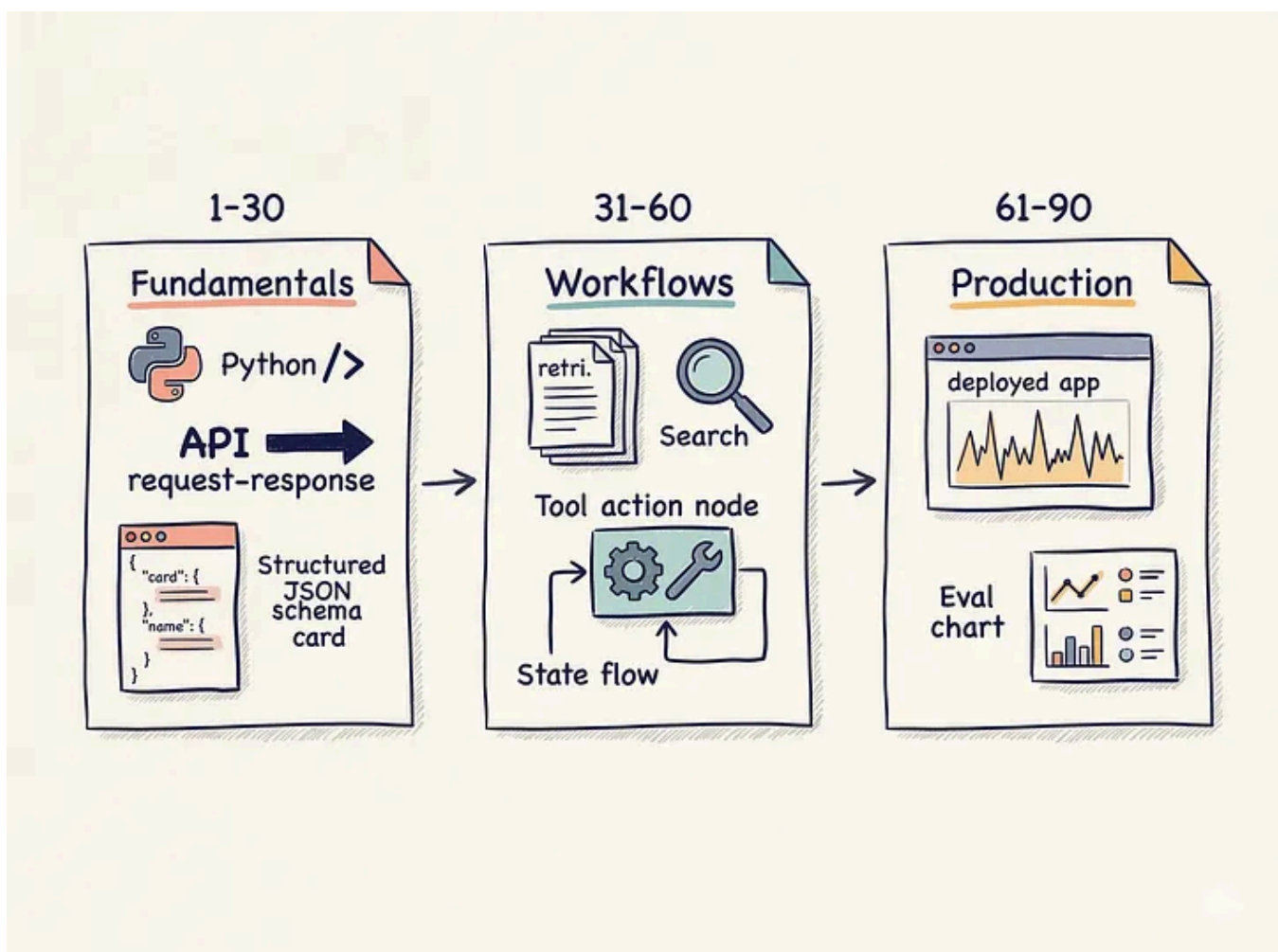
Your focus shifts to retrieval, tool calling, and workflow basics.

Your milestones are building one RAG project and one tool-using workflow. You should also write one document explaining the tradeoffs and failures you encountered. You will spend a lot of time debugging why your model decided to call the wrong tool. This is totally normal. You will learn to write better docstrings for your Python functions so the model understands them better.

Days 61 to 90

Your focus is entirely on evaluations, deployment, observability, and reliability.

Your milestones are deploying one project to a real server. You need to add a basic evaluation loop to your code. You must publish a serious portfolio README or a blog post explaining exactly what you built and why.



This section matters because it gives you boundaries. It stops you from spending 90 days just watching YouTube tutorials. It forces you to type code into your editor and watch it fail.

The Skill Matrix

I have broken down the skills into a matrix. This helps you stop trying to learn everything at once.

Must Have

Python and basic software engineering. You cannot skip this. APIs and backend basics like HTTP and JSON. Prompting and structured outputs. Communication and writing clear documentation.

Learn Soon

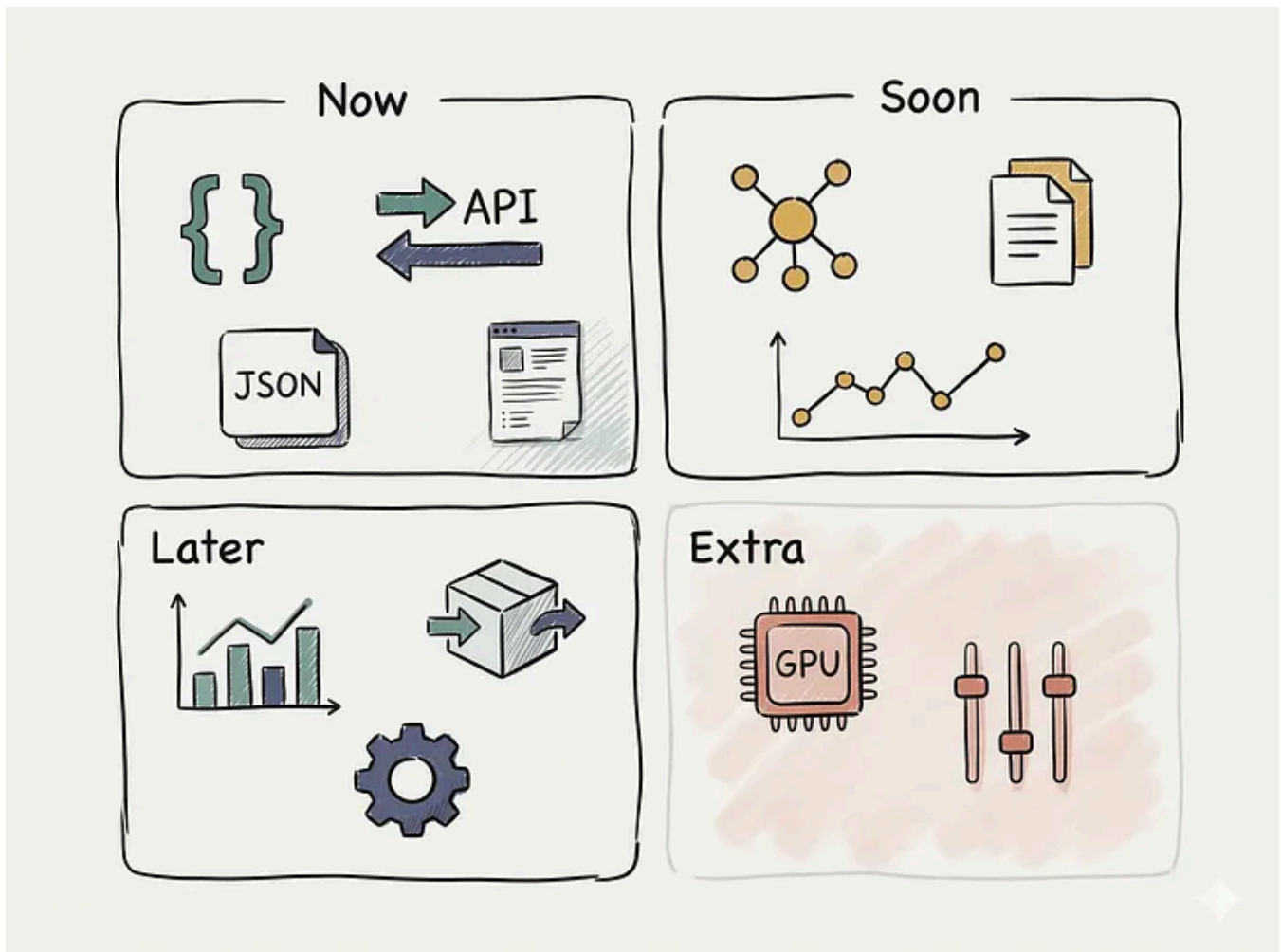
Tool calling and basic workflows. RAG and retrieval concepts. Basic observability and debugging API traces. You need to see exactly what prompt was sent to the model.

Learn Later

Evaluations and LLM-as-a-judge patterns. Deployment and ops for AI systems. Product thinking and user experience under failure.

Nice To Have

Fine-tuning models. Deep GPU optimization. Custom embedding training. You almost never need to do this in your first year.



How I Would Measure Progress

You have to be honest with yourself about your progress. Reading articles does not count as progress.

There are bad signals you should watch out for. The number of courses you have completed is a bad signal. The number of bookmarks you have saved is meaningless. The number of frameworks you have pip-installed and abandoned means nothing.

You need to look at better signals. Measure the number of projects you have actually shipped. Look at how fast you can debug a failure when an API call crashes. Ask yourself if you can explain your architecture clearly to another developer. Can you measure the quality, latency, and cost of your system? Can other people actually use your project and understand it without you standing next to them?

Progress is not saying you understand the concept. Progress is saying you can build the thing, measure its performance, and explain exactly what breaks under load.

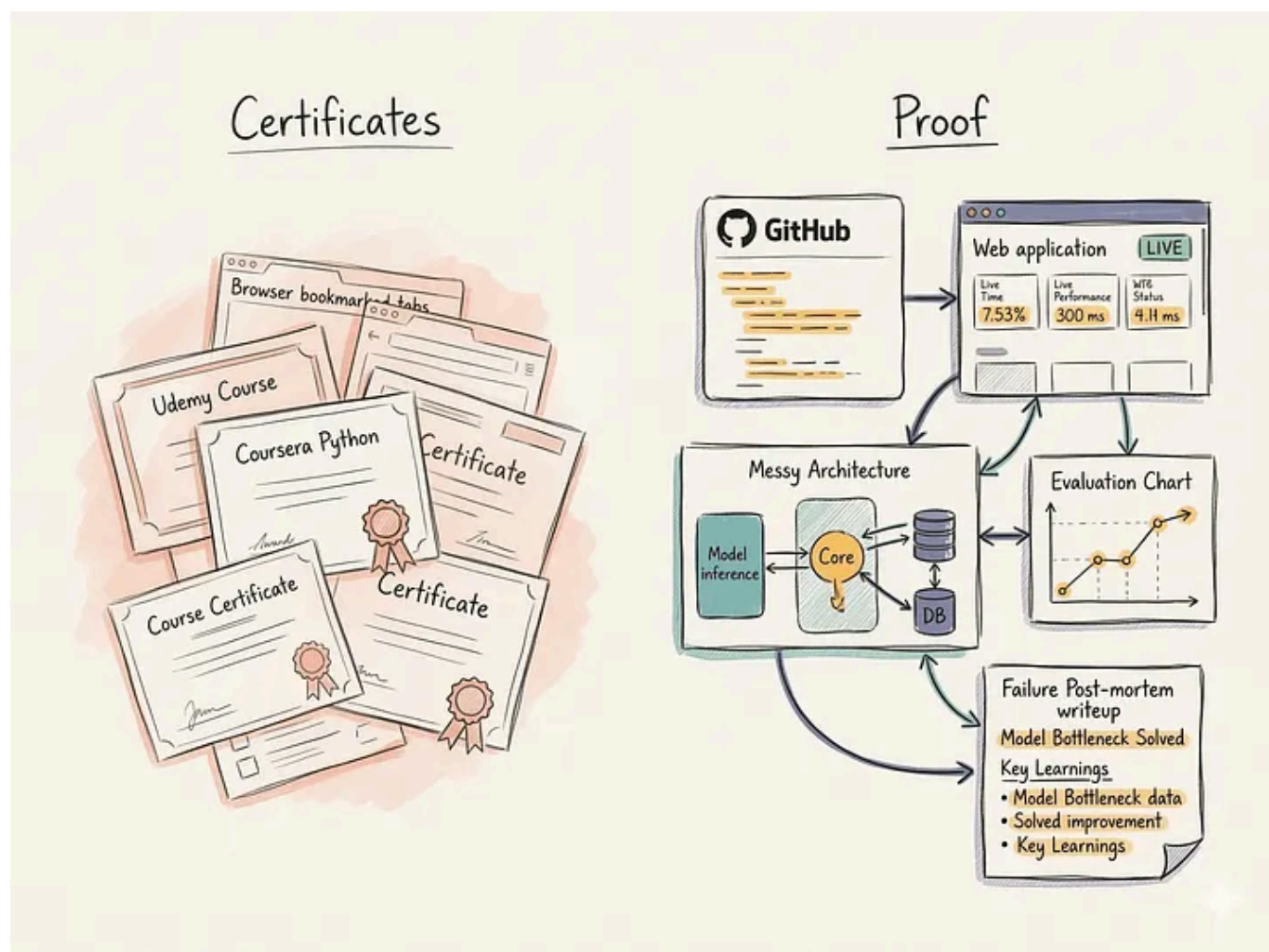


Portfolio Advice: What To Show Instead Of Certificates

Hiring managers do not care about your completion certificate from an online course. What actually signals skill is a clean GitHub repository. They want to see a deployed demo they can click on, or architecture diagrams that show you thought about the system design. They want to see evaluation results. If you show them a chart where your prompt iteration improved the pass rate from 60% to 92%, they will probably interview you.

They want to read failure analysis writeups. Tell them about the time your agent got stuck in an infinite loop and burned five dollars in API credits. Tell them how you fixed it by adding a max-iteration counter and a fallback response. That shows how you actually built the thing and felt the pain of operating it.

A strong portfolio project with a good writeup is worth more than a pile of certificates. Your README file is your resume. It should explain the problem you solved. It should list the models you used and why you chose them. It should explicitly state the latency and cost per run. If you build a document summarizer, tell me it costs three cents and takes four seconds to run. That is engineering context.

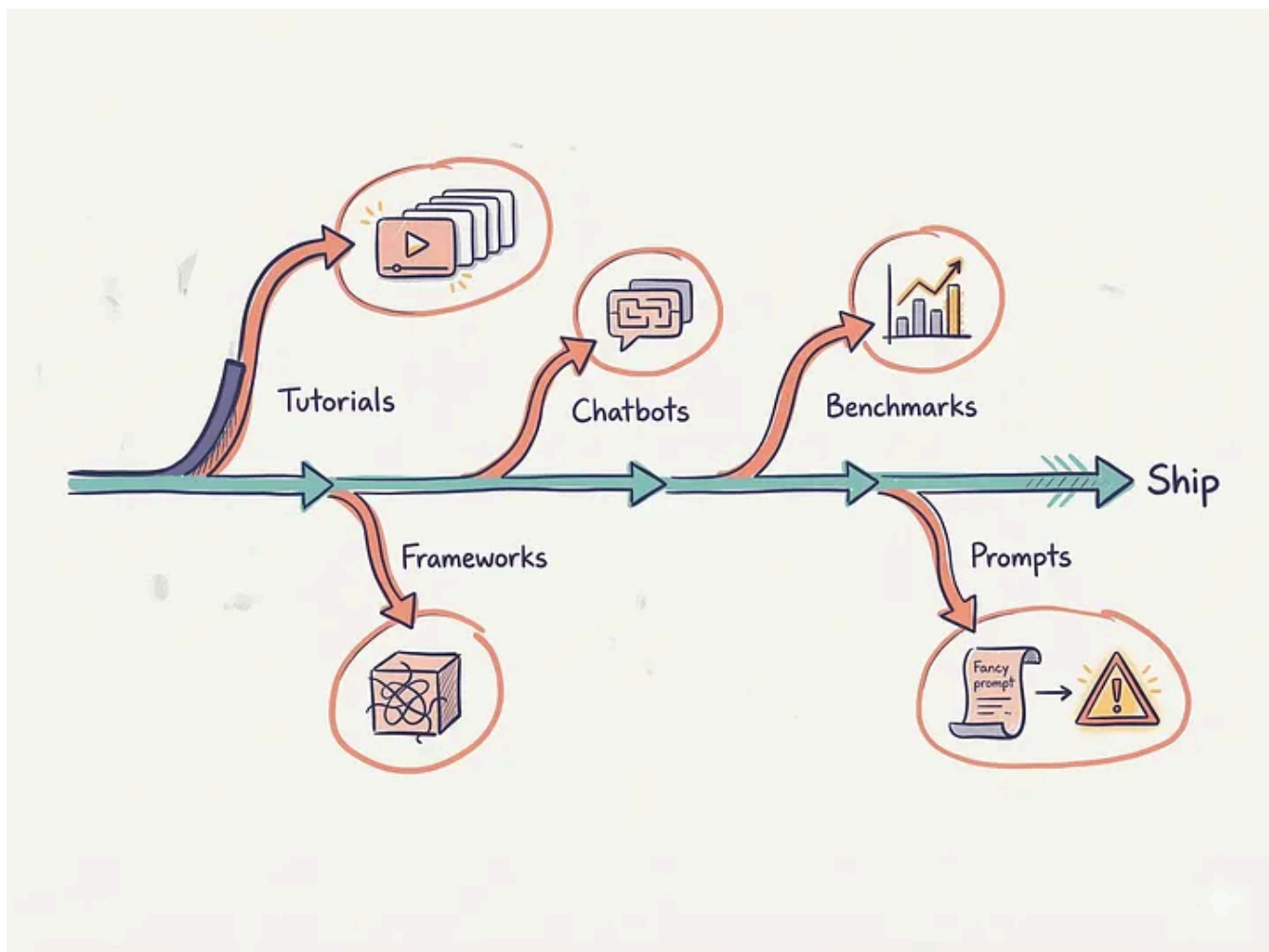


Common Traps I Would Avoid

1. **Taking courses forever without shipping anything.** You get stuck in tutorial hell. You watch someone else code and you feel like you are learning. You are not. You are just watching television. Close the browser and open your terminal.
2. **Learning frameworks before fundamentals.** Do not learn CrewAI or AutoGen before you know how to make a basic API call with the standard OpenAI SDK. Frameworks hide the plumbing. You need to understand the plumbing so you can fix it when it breaks.
3. **Building only chatbots.** Chatbots are the easiest thing to build and the hardest thing to evaluate. Build background workers. Build data extraction pipelines.

Build things that do not have a chat interface.

4. **Never learning evaluations or debugging.** If you just eyeball the output and say “looks good to me”, you are not doing engineering. You are doing vibes-based development. You need a test suite.
5. **Obsessing over model benchmarks instead of building.** I honestly do not care if Claude 4.6 Opus scores two points higher on a math test than GPT-5.3 Codex. It does not matter for 99% of business applications. Pick a good model and start writing code.
6. **Calling every workflow an agent.** If you have a script that runs three steps in a strict sequence, that is a pipeline. It is not an agent. Stop using hype words for simple code.
7. **Ignoring product and UX.** If your AI feature takes twenty seconds to load and you do not show a loading spinner, your users will leave. The user experience matters just as much as the prompt.
8. **Mistaking prompt cleverness for system reliability.** You can write the most beautiful, complex prompt in the world. It will still fail randomly. You need system-level reliability. You need retries, fallback models and exception handling.



The Meta-Rule: Build, Measure, Ship

This is the only philosophy you need. Learn enough theory to avoid magical thinking. You need to know what a token is. You need to know that models do not actually think, they just predict the next word based on probabilities.

Learn enough engineering to stop being blocked. You need to know how to install packages, set up virtual environments, and push code to a server.

Build enough projects to become credible. One good project is better than ten bad ones. Measure enough failures to become trusted. The best engineers I know are the ones who can tell you exactly how their system will break before it even happens.

Building AI systems is incredibly fun once you get past the initial wall of confusion. Stop reading about the future of artificial general intelligence. Open your editor, paste in your API key, and write your first script.

Continue Reading

[AI Agents Explained \(2026\): What They Really Are and How to Build Them](https://medium.com/@anubhavgoyal101/c80bd754c753) — The best immediate follow-up because this roadmap repeatedly distinguishes real agent systems

from hype, and this article sharpens that mental model.

From Basic RAG to Advanced Retrieval: A Practical Roadmap Using the Modern RAG Stack — A natural next step after Stage 3 because it turns the roadmap's RAG advice into a deeper implementation guide.

LangGraph AI Agents Explained: Building Reliable LLM Agents with State Machines — Extends the workflow and state-machine ideas from the roadmap into a concrete agent architecture.

Data Science

Artificial Intelligence

Programming

Python

Machine Learning



Follow

Published in Data Science Collective

921K followers · Last published 1 day ago

Advice, insights, and ideas from the Medium data science community



Follow

Written by Anubhav

689 followers · 28 following

I write deep-dives on agents, RAG, evals, and what actually breaks when you ship AI systems

